



[Sắp Xếp - Tìm Kiếm]. Bài 23. Sắp xếp theo tần suất

[C++ CB NC](#)[C++ NC](#)[DSA](#)[JAVA OOP](#)[View
as PDF](#)

Cho mảng **A[]** gồm **n** phần tử. Bạn hãy thực hiện làm các công việc sau đây :

- Sắp xếp các phần tử trong mảng theo tần suất giảm dần, nếu 2 số có cùng tần suất thì số nào nhỏ hơn sẽ in ra trước.
- Sắp xếp các phần tử trong mảng theo tần suất giảm dần, nếu 2 số có cùng tần suất thì số nào xuất hiện trước sẽ in ra trước.

Gợi ý : Dùng map để đếm tần suất rồi đưa ra vector<pair<int, int>> để sắp xếp. Không nên viết comparator dựa vào tần suất thông qua map vì sẽ bị sai ở điều kiện thứ 2. Ví dụ mảng {1, 2, 2, 1, 3, 4, 4} thì nó sẽ in {1, 2, 2, 1, 4, 4, 3} nhưng ở đây bạn cần in các giá trị giống nhau đứng cạnh nhau.

Đầu vào

Dòng đầu tiên đưa vào **n** là số phần tử của mảng **A[]**

Dòng tiếp theo là **n** số **A[i]**

Các số được viết cách nhau một vài khoảng trống.

Giới hạn

$$1 \leq n \leq 10^5$$

$$0 \leq A[i] \leq 10^6$$

Đầu ra

Dòng đầu tiên in ra các số được sắp xếp theo yêu cầu 1.

Dòng thứ 2 in ra các số được sắp xếp theo yêu cầu 2.

Ví dụ :

[Submit solution](#)[All submissions](#)[Best submissions](#)

✓ **Points:** 1.00 (partial)

🕒 **Time limit:** 1.0s

📄 **Memory limit:** 256M

💻 **Input:** stdin

🖨️ **Output:** stdout

📝 **Problem source:**
28Tech

➤ **Problem type**

▼ **Allowed languages**
C, C#, C++, Java, Kotlin,
Pascal, PyPy, Python,
Scratch

10

6 8 4 10 3 4 10 2 4 1

Copy

Output 01

4 4 4 10 10 1 2 3 6 8

4 4 4 10 10 6 8 3 2 1

Copy

Input 02

10

2 5 6 7 2 1 3 10 3 6

Copy

Output 02

2 2 3 3 6 6 1 5 7 10

2 2 6 6 3 3 5 7 1 10

Copy

Comments

Report an issue

Please read the [guidelines](#) before commenting.

^

1

v



commented 12:51:59 pm, 11/09/2025

nmtuong_cpp

Ý 1:

Đầu tiên là dùng một map để lưu cặp key value là giá trị và tần suất.



map<key, value> freqMap; // Tạo map tần suất

Nên tạo thêm một vector có phần tử là pair, push_pack các cặp key - value của map vừa tạo.

Rồi xử lý hàm comparator thôi.

Xử lý trường hợp dấu "=" xảy ra trước. Nếu điều kiện đầu bằng nhau thì xử lý điều kiện thứ 2, rồi return về.

Trường hợp dấu "=" không xảy ra thì trực tiếp return kết quả của điều kiện đầu.

Ý 2:

Lúc đầu mình dùng stable_sort nhưng kết quả không được :)))

Hỏi em chatbot thì em bảo sai struct với value - frequency - firstOccurence sẽ dễ xử lý hơn.

Còn không thì phải có một biến toàn cục là map<int, int> glbFirstOccurence đặt ở trước hàm comparator cho ý 2. Tức là khi 2 <tần suất> cùng bằng nhau, thì phải dựa vào <giá trị> đính kèm với <tần suất> để truy ngược ra lại <giá trị> đính kèm với <lần xuất hiện đầu tiên>.

Link source code của mình:

<https://pastebin.com/wW9esw7Y>



Hello, Nguyễn Bảo Toàn.

Post!

Được phát triển bởi **28TECH.COM.VN**